

QuickStock

Resumo Completo do Projeto

Mobile · Backend · Banco de Dados
Material de estudo para apresentação

Gerado em: 14 de junho de 2026

O QuickStock é um sistema de gestão de estoque e marketplace B2B para empresas do setor de bebidas (bares, restaurantes, revendas). O projeto é composto por um app mobile (React Native/Expo) e uma API REST (Spring Boot/PostgreSQL). Este documento resume arquitetura, modelos conceitual/lógico/físico, principais códigos e fluxos de negócio.

1. Visão geral do sistema

1.1 O que o sistema faz

- Cadastro e login de empresas compradoras (CNPJ, telefone, usuário admin)
- Marketplace B2B: navegar fornecedores, ver produtos, adicionar à sacola e finalizar compra
- Acompanhamento de pedidos estilo delivery (enviada !' confirmada !' em rota !' entregue)
- Gestão de catálogo próprio: CRUD de produtos com imagem e estoque
- Barraquinhas/filiais com estoque por ponto de venda (fluxo PDV legado)
- Dashboard financeiro: resumo de compras/vendas e stock do dia
- Endereços de entrega, formas de pagamento e cartões salvos

1.2 Repositórios

Projeto	Tecnologia / Path
Mobile (front)	React Native 0.83 + Expo 54 — c:\Dev\Code\Faculdade\Mobile
Backend (API)	Spring Boot 3.4 + Java 17 — c:\Dev\Code\Faculdade\QuickStock-BackEnd
Banco de dados	PostgreSQL — banco quickstock, porta 5432
Comunicação	HTTP REST JSON — API na porta 8080

1.3 Stack tecnológica

Camada	Tecnologia	Detalhe
Mobile	Expo / RN	TypeScript, React Navigation, AsyncStorage, expo-image-picker
Backend	Spring Boot	JPA/Hibernate, Lombok, jjwt, springdoc-openapi
Banco	PostgreSQL	ddl-auto=update + data.sql idempotente
Auth	JWT manual	BCrypt para senha; token 24h; Bearer em /api/usuarios/me
Uploads	Multipart	Imagens em uploads/produtos/ servidas em /uploads/**

2. Modelo conceitual

O modelo conceitual descreve as entidades de negócio e seus relacionamentos, independente de implementação técnica.

2.1 Entidades principais

Entidade	Descrição conceitual
----------	----------------------

Empresa	Organização cadastrada (compradora ou distribuidora). Identificada por CNPJ.
Usuário	Pessoa que opera o sistema em nome de uma empresa. Possui perfil (Admin, etc.).
Produto	Item do catálogo (nome, preço, unidade, imagem, estoque, código único).
Fornecedor	Empresa do tipo DISTRIBUIDOR que vende no marketplace.
Solicitação de Compra	Pedido B2B: comprador solicita produtos a um fornecedor.
Pedido (financeiro)	Registro de venda: PDV na barraca ou checkout marketplace.
Endereço de Entrega	Local onde a mercadoria será recebida.
Forma de Pagamento	Preferência salva (PIX, crédito, débito, dinheiro).
Cartão Salvo	Dados mascarados do cartão (sem CVV).
Evento / Barraca	Contexto de feira/evento com ponto de venda físico (PDV).
Estoque Barraca	Quantidade de produto disponível em uma barraca específica.

2.2 Relacionamentos conceituais

Empresa COMPRADORA % % realiza % % % Solicitação de Compra % % fornece % % DISTRIBUIDORA

Solicitação de Compra % % contém % % % Item (produto + quantidade + preço)

Solicitação de Compra % % gera % % % Pedido marketplace + Pagamento

Empresa % % possui % % % Produtos (catálogo próprio ou recebidos na entrega)

Empresa % % cadastra % % % Endereços, Formas de Pagamento, Cartões

Evento % % contém % % % Barracas % % registram % % % Pedidos PDV + Estoque

2.3 Regras de negócio centrais

- Comprador só vê fornecedores do tipo DISTRIBUIDOR (exceto a própria empresa)
- Checkout debita estoque do fornecedor imediatamente
- Na entrega, estoque é creditado ao comprador (produto copiado com codigo_origem)
- Sacola multi-fornecedor gera uma solicitação por distribuidora + taxa R\$ 7/fornecedor
- Tracking de pedido é simulado por tempo (demo): ~20s em rota, ~55s entregue
- Dois estoques coexistem: Produto.estoque (empresa) e EstoqueBarraca (PDV por barraca)

3. Modelo lógico

O modelo lógico traduz o conceito em estruturas de dados e contratos de API.

3.1 Modelo lógico de dados — 15 tabelas

Grupo	Tabelas	Função
Autenticação	perfis, empresas, usuarios	Cadastro, login, perfis de acesso
Catálogo	produtos	Produtos por empresa + estoque + código
PDV	eventos, barracas,	Vendas em evento/feira
Marketplace	estoque_barraca_pedido, solicitações_compra, itens_pedido_pagamentos	Compras B2B entre empresas
Cadastros aux.	endereços_entrega, formas_pagamento_salvas, cartoes_pagamento_salvos	Checkout e preferências

3.2 Modelo lógico de API — grupos de endpoints

Prefixo /api/...	Responsabilidade
cadastro, usuarios/login, usuarios/	

me

marketplace/fornecedores	Listar distribuidoras e catálogo
solicitacoes-compra	Checkout, histórico, acompanhamento
produtos (+ /upload)	CRUD catálogo próprio + imagens
barracas, estoque-barraca	Barraquinhas e estoque PDV
enderecos, formas-pagamento, cartões-pagamento, financeiro/resumo, financeiro/stock-dia	Dados de checkout
pedidos, itens-pedido, pagamentos	Dashboard e carteira
empresas, eventos, perfis, notificacoes	Fluxo PDV (legado)
	Cadastros e alertas

3.3 DTOs e contratos principais (Backend)

- CadastroContaRequestDTO !' POST /api/cadastro (empresa + usuário num payload)
- LoginRequestDTO !' LoginResponseDTO (token JWT + dados do usuário)
- SolicitacaoCompraRequestDTO !' itens[], enderecoEntregald, metodoPagamento, empresaCompradoraId
- SolicitacaoCompraResponseDTO !' status, previsaoEntrega (calculado), itens, endereço snapshot
- ProdutoRequestDTO !' nome, precoVenda, unidade, estoque, imageUrl, empresald

3.4 Modelo lógico do Mobile — camadas

Screens (UI) !' Contexts (estado) !' Services (HTTP) !' Backend API
 Persistência local: AsyncStorage (sessão, carrinho, chaves PIX, endereço selecionado)

4. Modelo físico

4.1 Infraestrutura

Componente	Implementação física
Servidor API	JVM local — Spring Boot em localhost:8080
Banco	PostgreSQL local — jdbc:postgresql://localhost:5432/quickstock
App mobile	Expo Go ou build nativo — conecta via IP da máquina (não localhost no device)
Arquivos estáticos	Disco: QuickStock-BackEnd/uploads/ exposto em /uploads/**
Sessão mobile	AsyncStorage @quickstock_session (token + user + expiração)
Carrinho mobile	AsyncStorage @quickstock_purchase_cart_{empresald}

4.2 Configuração física do backend (application.properties)

- spring.jpa.hibernate.ddl-auto=update — schema evolui com entidades JPA
- spring.sql.init.mode=always — data.sql roda a cada startup
- jwt.secret + jwt.expiration-ms=86400000 (24 horas)
- upload.dir=uploads/produtos — limite 10MB por arquivo

4.3 Configuração física do mobile (api.ts)

- API_BASE_URL = http://{host}:8080
- Android emulator: host 10.0.2.2 | Device físico: IP da rede local
- getImageUrl() prefixa paths relativos (/uploads/...) com a base da API

4.4 Seeds e inicialização

- data.sql — migrações idempotentes + 3 distribuidoras + 15 produtos oficiais
- EnderecoSeedRunner — endereços demo por empresa compradora
- FormaPagamentoSeedRunner — PIX/crédito/débito/dinheiro demo

- ProdutoCatalogoFixRunner — IDs fixos 1001–1025, códigos MKT-*
- MarketplaceDataFixRunner — sincroniza catálogo e desativa legado

5. Backend — arquitetura e código principal

5.1 Estrutura de pacotes

```
com.quickstock.backend
  controller/ — 18 REST controllers (@RestController, /api/*)
  service/ — 14 services (regras de negócio)
  entity/ — 15 entidades JPA (@Entity)
  repository/ — 15 interfaces Spring Data JPA
  dto/ — 33 DTOs request/response
  config/ — WebConfig (CORS, uploads) + ApplicationRunners
  exception/ — CadastroException + GlobalExceptionHandler
```

5.2 Controllers mais importantes

Controller	Path base	Função
CadastroController	/api/cadastro	Registro empresa + admin
UsuarioController	/api/usuarios	Login, /me (JWT), CRUD
MarketplaceController	/api/marketplace	Fornecedores e produtos
SolicitacaoCompraController	/api/solicitacoes-compra	Checkout B2B
ProdutoController	/api/produtos	CRUD + upload imagem
FinanceiroController	/api/financeiro	Resumo e stock-dia
BarracaController	/api/barracas	Barraquinhas + estoque
EnderecoEntregaController	/api/enderecos	Endereços + ViaCEP

5.3 Services — métodos críticos

CadastroService.cadastrarConta(dto)

- Cria Empresa (tipo COMPRADOR) + Usuario com perfil Admin + senha BCrypt

JwtService

- generateToken(Usuario) — HMAC-SHA, claim sub=userId

- isValidToken(token), getUserId(token) — usado em GET /api/usuarios/me

SolicitacaoCompraService.criar(dto) — coração do checkout

- 1. Valida comprador, fornecedor DISTRIBUIDOR, usuário, pagamento
- 2. Valida enderecoEntregaId pertence à empresa compradora
- 3. Para cada item:

EstoqueProdutoService.debitarEstoque(produto, qtd)

- 4. Cria SolicitacaoCompra (status aguardando_liberacao)

- 5. criarPedidoMarketplace() !' Pedido tipo=marketplace + ItemPedido + Pagamento

- 6. Vincula solicitacao.pedido

SolicitacaoCompraService.obterPorId / listarPorComprador

- atualizarStatusDemonstracao() — avança status por tempo (demo)

- Se entregue !' `EstoqueProdutoService.creditarCompradorSeNecessario()`
`EstoqueProdutoService`
 - `debitarEstoque(produto, qtd)` — subtrai estoque do fornecedor; erro 400 se insuficiente
 - `creditarCompradorSeNecessario(solicitacao)` — copia produtos para catálogo do comprador
 - `gerarCodigoCatalogo(empresald)` — gera EMP-{id}-{seq}
- `MarketplaceService`
- `listarFornecedores(empresaCompradorald)` — tipo DISTRIBUIDOR, exclui própria empresa
 - `listarProdutosFornecedor(id)` — produtos ativos com estoque
- `FinanceiroService`
- `obterResumo(empresald)` — compras (solicitações) vs vendas (pedidos PDV) 6 meses
 - `obterStockDia(empresald)` — movimentos do dia + margem estimada

5.4 Entidades JPA principais

Entidade	Tabela
Empresa	empresas
Produto	produtos
SolicitacaoCompra	solicitacoes_compra
Pedido	pedido
CartaoPagamentoSalvo	cartoes_pagamento_salvos

Destaque

tipo: COMP
codigo UNIC
Snapshot e
tipo pdvima
Dados mas

Pasta	Conteúdo
screens/	Telas: Home, Cart (marketplace), Sacola, Login, Register, etc.
components/	UI reutilizável: BottomTabBar, ProductCard, modais, headers
context/	Estado global: Auth, PurchaseCart, Products, Barraquinhas
services/	Chamadas HTTP e AsyncStorage
config/	api.ts — base URL e getImageUrl
theme/	theme.ts — cores, spacing, fonts
utils/	safeArea, pixUtils, cep, imagens, datas

6.2 Navegação (App.tsx)

Dois stacks controlados por AuthContext:

- GuestNavigator: Welcome !' Register | Login
- AuthenticatedNavigator: Home, Barraquinhas, Cart, Sacola, FormasPagamento, Cards, + telas secundárias
- Tab bar customizada (BottomTabBar.tsx) — não usa `@react-navigation/bottom-tabs`
- Botão central "+" !' `ManageProductsScreen` (AddItem)

6.3 Contextos — estado global

Context	Hook	Responsabilidade
AuthContext	<code>useAuth()</code>	user, token, signIn, signOut, updateUser — AsyncStorage
PurchaseCartContext	<code>usePurchaseCart()</code>	Sacola: addItem, grupos por fornecedor, taxa R\$7/ fornecedor
ProductsContext	<code>useProdutos()</code>	Catálogo próprio da empresa — refresh após entrega

BarraquinhasContext	useBarraquinhas()	CRUD barraquinhas
ConfirmDialogContext	useConfirmDialog()	Modal de confirmação reutilizável

6.4 Services — integração com API

Service	Endpoints principais	Uso
authService	POST /cadastro, /usuarios/login, GET /me	Login e registro
marketplaceService	GET fornecedores, POST solicitacoes-compra	Marketplace e checkout
productService	CRUD /produtos, POST /upload	Gestão catálogo próprio
enderecoService	GET/POST /enderecos, GET /cep/{cep}	Endereços na sacola
formaPagamentoService	GET/POST /formas-pagamento	Tipos de pagamento
cartaoPagamentoService	GET/POST /cartoes-pagamento	Cartões mascarados
financeiroService	GET /financeiro/resumo, /stock-dia	Home e Carteira
barracaService	CRUD /barracas, PUT /estoque	Barraquinhas
pixChaveService	AsyncStorage local	Chaves PIX (não vai ao backend)
purchaseCartStorage	AsyncStorage por empresaId	Persistência da sacola

6.5 Telas principais e funções-chave

Tela / Arquivo	Função	Código importante
WelcomeScreen	Landing login/cadastro	Navega para Login ou Register
RegisterScreen	Cadastro unificado	cadastrarConta() — formata CNPJ/telefone
Login.tsx	Autenticação	signIn() !' authService.login()
HomeScreen	Dashboard	ProductStockCard, FinancialDonutChart, buscarResumoFinanceiro
CartScreen	Marketplace (NÃO é carrinho)	listarFornecedores, pedidos recentes
StoreVitrineScreen	Vitrine fornecedor	listarProdutosFornecedor !' ProductDetail
ProductDetailScreen	Detalhe + add sacola	addItem() do PurchaseCartContext
SacolaScreen	Carrinho real + checkout	processarCheckout() !' criarSolicitacaoCompra por fornecedor
PedidoAcompanhamentoScreen	Tracking pedido	buscarSolicitacao() polling 5s
ManageProductsScreen	CRUD produtos	ProductFormModal + uploadImagemProduto
BarraquinhasScreen	CRUD barraquinhas	BarracaFormModal + sincronizar estoque
CardsScreen	Carteira financeira	buscarStockDia, buscarResumoFinanceiro
FormasPagamentoScreen	Pagamentos salvos	Modais PIX e cartão
EnderecosScreen	Endereços entrega	EnderecoFormModal + consultarCep
ConfiguracoesScreen	Perfil e empresa	atualizarUsuario, atualizarEmpresa

6.6 Componentes reutilizáveis importantes

- BottomTabBar — navegação inferior com badge na Sacola
- TabScreenLayout — layout padrão com safe area e tab bar
- ProductCard / ProductStockCard — card de produto com imagem, preço, estoque
- CheckoutPaymentModal — modal PIX/cartão/dinheiro no checkout
- ProductFormModal — formulário criar/editar produto com picker de imagem

- ScreenTopGradient — gradiente amarelo (#F8B125) no topo das telas autenticadas

6.7 Tema visual

- Cor principal autenticado: #F8B125 (amarelo/dourado)
- Telas auth: gradiente azul !' laranja (Background.tsx)
- Tokens em theme.ts: primaryYellow, primaryBlue, SPACING, FONTS
- Safe area Android: useBottomInset() mínimo 48px (utils/safeArea.ts)

7. Banco de dados — resumo

7.1 Evolução do schema

Fase	Alteração
Original	10 tabelas: PDV, eventos, barracas, produtos, usuários
Marketplace	+4 tabelas: endereços, formas pagamento, solicitações + itens; empresas.tipo
Checkout	pedido estendido (tipo marketplace); pagamentos.referencia_pagamento
Estoque	produtos.estoque, codigo, codigo_origem; solicitacoes.estoque_comprador_creditado
Cartões	cartoes_pagamento_salvos; 3 distribuidoras parceiras; legado INATIVO

7.2 Distribuidoras ativas no marketplace

Loja	CNPJ	Produtos (códigos)
Casa dos Vinhos	30.001.001/0001-01	MKT-CDV-001 a 005 (IDs 1001–1005)
Cervejaria Caruaru	30.002.002/0001-02	MKT-CC-001 a 005 (IDs 1011–1015)
Whisky Labs	30.003.003/0001-03	MKT-WL-001 a 005 (IDs 1021–1025)

7.3 Status de solicitação de compra

- enviada / aguardando_liberacao !' confirmada !' em_rota !' entregue | cancelada
- Na entregue: credita estoque ao comprador via EstoqueProdutoService

8. Fluxos principais (ponta a ponta)

8.1 Cadastro e login

Mobile: RegisterScreen !' POST /api/cadastro !' CadastroService.cadastrarConta
!' cria empresas + usuarios (BCrypt) !'
usuário faz login manual
Mobile: Login !' POST /api/usuarios/login !'
JWT !' AsyncStorage !' AuthContext
Sessão: GET /api/usuarios/me com Bearer token valida expiração

8.2 Compra no marketplace (fluxo completo)

1. CartScreen !' GET /marketplace/fornecedores
2. StoreVitrineScreen !' GET /marketplace/fornecedores/{id}/produtos
3. ProductDetailScreen !'


```
PurchaseCartContext.addItem() !' AsyncStorage
4. SacolaScreen !' agrupa por fornecedor,
seleciona endereço e pagamento
5. CheckoutPaymentModal !' processarCheckout()
!' para cada fornecedor: POST /
solicitacoes-compra
!' Backend: debitarEstoque + criar
SolicitacaoCompra + Pedido marketplace
6. PedidoAcompanhamentoScreen !' GET /
solicitacoes-compra/{id} (polling 5s)
7. Status entregue !'
creditarCompradorSeNecessario !'
refreshProdutos() na Home
```

8.3 Gestão de produtos próprios

```
ManageProductsScreen !' ProductsContext !' GET /
api/produtos?empresaId=
ProductFormModal !' POST/PUT /api/produtos +
POST /api/produtos/upload
ProdutoController !'
EstoqueProdutoService.gerarCodigoCatalogo()
se novo
```

8.4 Barraquinhas (PDV)

```
BarraquinhasScreen !' POST /api/barracas !'
BarracaService.criar
!' cria Evento "Operação principal" se
necessário
PUT /api/barracas/{id}/estoque !' sincroniza
EstoqueBarraca
```

8.5 Dashboard financeiro

```
HomeScreen / CardsScreen !' GET /api/
financeiro/resumo
!' FinanceiroService agrega
SolicitacaoCompra (compras) vs Pedido PDV
(vendas)
GET /api/financeiro/stock-dia !' movimentos e
margem do dia
```

9. Diagrama integrado Mobile !" API !" Banco

```
% % % % % % % % % % % % % % HTTP/JSON % % % %
% App Mobile % % % % % % % % % % % % % % % %
PostgreSQL %
% (Expo/RN) % :8080 % 18
controllers % :5432 % quickstock %
% % % % % % % % % % % % % % % % % % % % % %
% % % % % % % % % % % % % % % % % % % % % %
% AsyncStorage uploads/
(imagens)
sessão, sacola, PIX local
```

9.1 Camadas por responsabilidade

Camada	Mobile	Backend
Apresentação	screens/ + components/	controllers/ (REST)
Estado/Regra	context/ + utils/	service/
Dados	services/ + AsyncStorage	repository/ + entity/
Persistência	AsyncStorage (local)	PostgreSQL + uploads/

10. Glossário e dicas para apresentação

10.1 Termos importantes

Termo	Significado
Marketplace B2B	Compra entre empresas (revenda compra de distribuidor)
Sacola	Carrinho de compras multi-fornecedor no mobile
Cart (tab)	Tela de marketplace — listagem de fornecedores (não é carrinho)
Solicitação de Compra	Entidade principal do pedido B2B no banco
codigo_origem	Rastreia produto recebido do fornecedor no catálogo do comprador
PDV	Ponto de venda — vendas em barraca/evento (fluxo original)
Snapshot endereço	Cópia do endereço gravada no pedido (não muda se endereço for editado)

10.2 Pontos fortes para destacar na apresentação

- Arquitetura em camadas clara: Mobile !' API REST !' PostgreSQL
- Dois fluxos de negócio coexistem: marketplace B2B + PDV em eventos
- Controle de estoque bidirecional: débito na compra, crédito na entrega
- Checkout multi-fornecedor com taxa de entrega por loja
- UX completa: cadastro, marketplace, sacola, pagamento, tracking, dashboard
- Persistência local inteligente (carrinho sobrevive ao fechar app)

10.3 Limitações conhecidas (transparência acadêmica)

- JWT validado apenas em /api/usuarios/me — demais rotas confiam no cliente
- Tracking de entrega simulado por timer (não há integração logística real)
- Chaves PIX ficam só no dispositivo (AsyncStorage), não no backend
- Schema evolui via Hibernate update (sem Flyway/Liquibase versionado)

10.4 Como demonstrar ao vivo

1. Registrar empresa !' login !' Home com gráfico financeiro
2. Tab Cart !' escolher fornecedor !' adicionar produtos à sacola
3. Sacola !' endereço + pagamento !' confirmar !' acompanhar pedido
4. Após entrega: ver produto creditado em "Meus produtos" (+)
5. Mostrar Swagger UI em <http://localhost:8080/swagger-ui.html>

